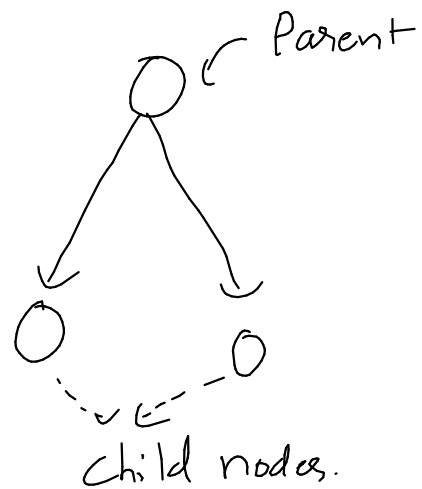
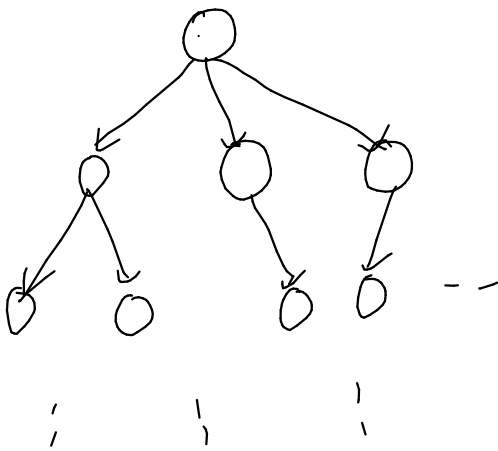


Trees → Trees are another type of data structure
Similar to arrays, Linked list.

→ Later we'll see that Trees are a special
Case of Graph data structure.

→ Tree is a group of nodes, linked from a single
Parent node to multiple child nodes, who in turn will
have child nodes of their own.

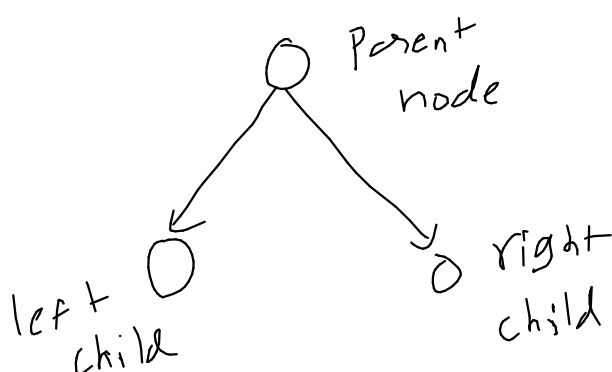


→ Similar to linked lists, every node contains a value
Segment to store some data.

→ We will mostly study about binary trees in this
course.

Binary tree

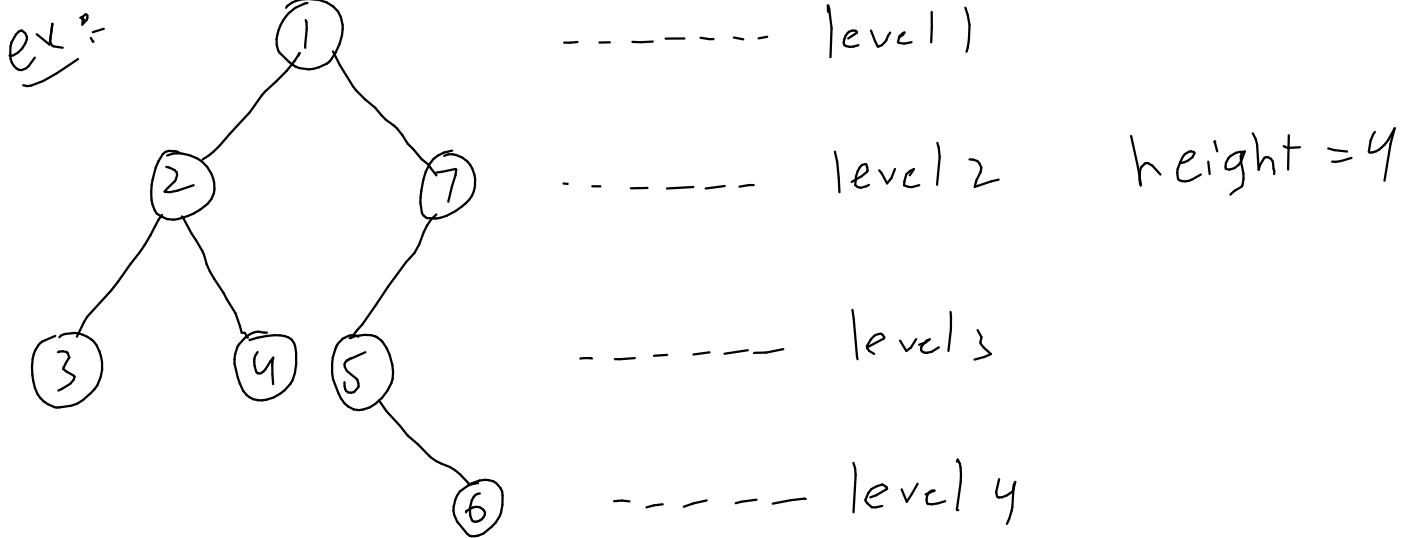
Every node contains 2 children
left child, right child.



(these child nodes can be
absent too. We will
just write them as
null).

Some basic terms on trees:-

- ① root :- root node is the topmost node in your tree, it doesn't have any parents.
- ② leaf nodes :- leaf nodes are the nodes with no child nodes, they appear at bottom.
- ③ height :- the number of levels your tree has.
- ④ interior (or) non-leaf nodes :- Any node which is not a leaf node.



root → node 1

leaf nodes → node 3, node 4, node 6

non-leaf nodes → 2, 7, 5

in this tree we can say,

left child of 1 is 2

right child of 1 is 7

left child of 7 is 5

right child of 7 is null

left, right of 6 is null

Every node contains a value, left, right segments.

Java implementations:-

Create a class for node like

this

```
class Node{
    int value;
    Node left;
    Node right;

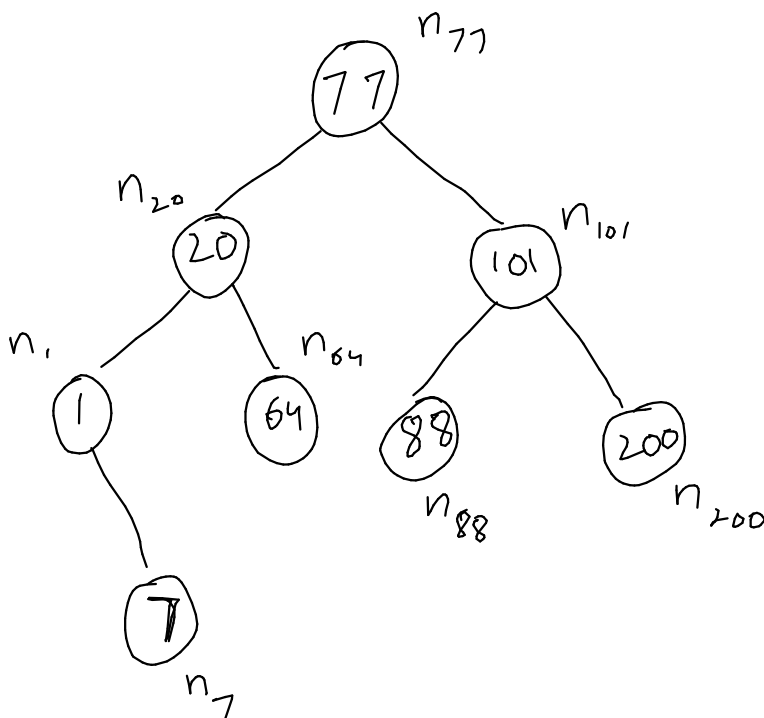
    Node(int val){
        value = val;
        left = right = null;
    }
}
```

• Every Node is a Node class object, can create object using

`Node n1 = new Node(6);`

This'll create a n1 object with value 6 and left, right initially pointing to null.

If you want to create a tree below, write this in your main.

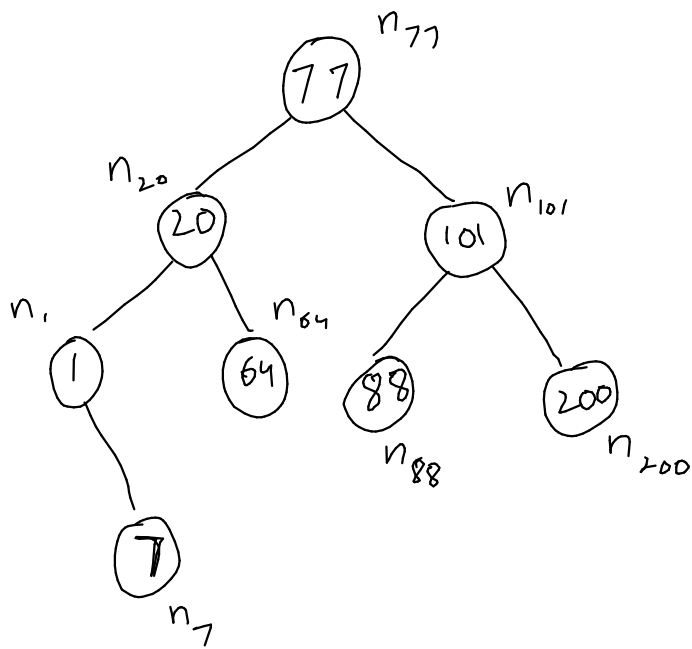


```
//Node n0 = new Node(0);
Node n1 = new Node(1);
Node n7 = new Node(7);
Node n20 = new Node(20);
Node n64 = new Node(64);
Node n77 = new Node(77);
Node n88 = new Node(88);
Node n101 = new Node(101);
Node n200 = new Node(200);
/*Node n9 = new Node(9);
Node n10 = new Node(10);
Node n11 = new Node(11);*/
```

```
// LINKING THE NODES IN THE TREE
n77.left = n20;
n77.right = n101;
n20.left = n1; n20.right = n64;
n1.right = n7;
n101.left = n88;
n101.right = n200;
```

→ You can directly change values at value; left, right data members in your node object.

How to travel between nodes :-



→ All coding questions on trees, they only give you the root node of your tree, similar to head node in Linked List.

if you have access to n_{77} object,

you can get n_{88} object using $n_{77}.right.left$,

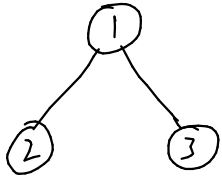
n_7 using $n_{77}.left.left.right$, n_{101} as $n_{77}.right$

We will use recursive functions heavily in trees, graphs. get used to it.

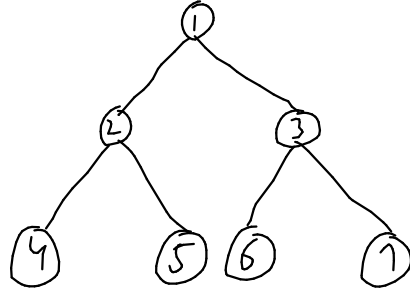
Perfect binary tree :- a tree where last level is completely filled.

ex:- Perfect binary tree of

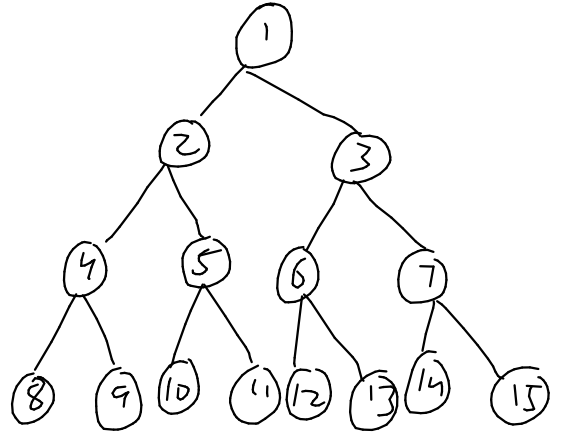
height 2



height 3



height 4



→ except for bottom most level, all nodes have exactly 2 children. and give values as 1, 2, --- in order. $\Rightarrow$ now you can call it a Perfect binary tree.

Some results on Perfect binary trees

① How many nodes will a Perfect binary tree of height 'h' contain?

level 1 $\rightarrow$ 1 node

level 2 $\rightarrow$ 2 nodes

level 3 $\rightarrow$ 4 nodes

$\vdots$

level n $\rightarrow 2 \times (\text{level}(n-1))$

$\rightarrow = 2^{n-1}$ nodes

every level contains twice the no. of nodes of its above level. Because every node will have 2 children in next level.

level h $\rightarrow 2^{h-1}$ nodes

total = $1 + 2 + \dots + 2^{h-1} = \boxed{2^h - 1}$ nodes

② What are the numbers of nodes at level a from top?

level 1 $\rightarrow 1$

level 2 $\rightarrow 2, 3$

level 3 $\rightarrow 4, 5, 6, 7$

level 4 $\rightarrow 8, 9, \dots, 15$

in general level ' a ' contains nodes $2^{a-1}, \dots, 2^a - 1$

Because level ' a ' ends with node $2^a - 1$ total no. of nodes in perfect binary tree of height a .
and level ' $a-1$ ' ends with node $2^{a-1} - 1$ (for $a > 1$)

$\Downarrow$
level ' a ' starts with $1 + (2^{a-1} - 1) = 2^{a-1}$

so 2^{a-1} to $2^a - 1$ numbered nodes in level ' a '.

③ given a Perfect binary tree of height h , and a node k , which level does node k lie on?

just find a suitable ' a ' such that

$$2^{a-1} \leq k \leq 2^a - 1$$

it will lie on level $a = 1 + \lfloor \log_2 k \rfloor$

④ what will be the number for left and right child of node k .

left child will be node $2k$

Right " " " $2k+1$.

→ just verify by looking at trees of different heights and at different nodes.

→ no proof needed. (already discussed in class)

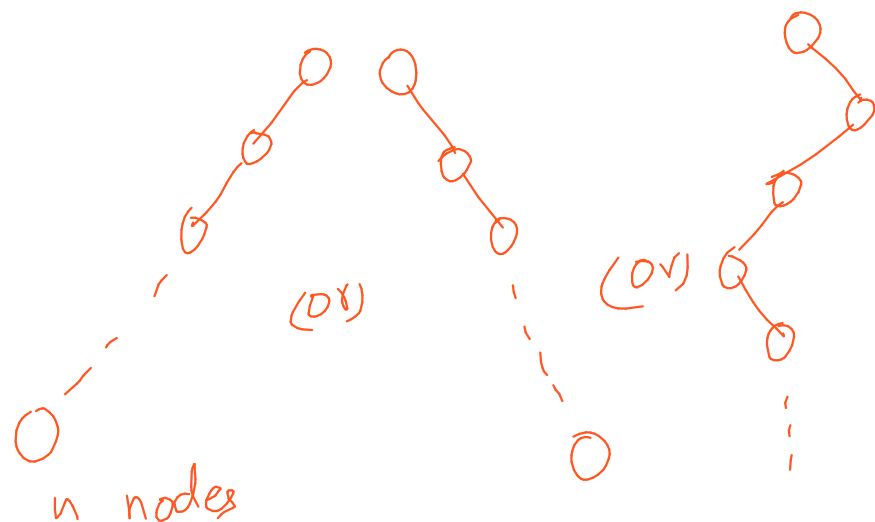
⑤ Given an arbitrary set of n nodes, you can construct any possible trees with them. of those trees,

(i) What is the max height of tree possible?

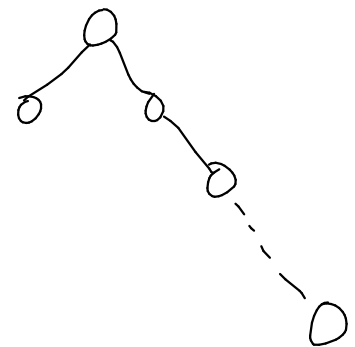
(ii) " " min " " " ?

(i) Max height possible is ' n '.

they look like

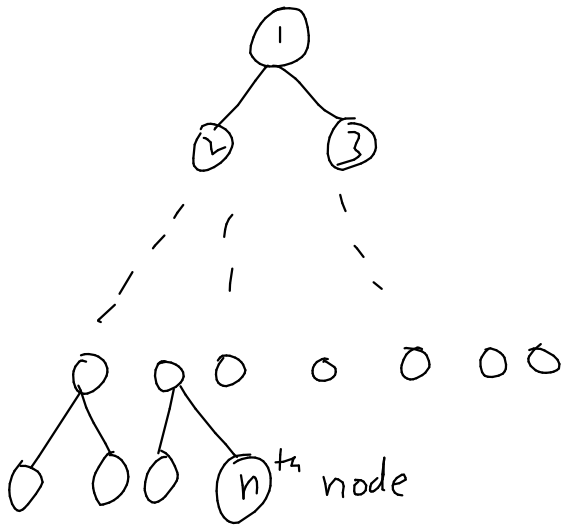


if you draw like



max height this can be is $(n-1)$.

(ii) Min possible height trees look like this, completely packed.



$\Rightarrow$ its height will be

$$\boxed{1 + \lfloor \log_2 n \rfloor} \quad (3^{\text{rd}} \text{ question})$$

and no smaller tree possible.

$\Rightarrow$ there can be multiple trees of same this height.